Introduction to Software Engineering

Project Proposal & Plan

The student team is required to complete the **Project Proposal & Plan** documentation for the assigned course project, following the attached template.



Software Engineering Department
Faculty of Information and Technology
University of Science

Table of Contents

Objectives 1
1 Member Contribution Assessment 2
2 Vision 3
2.1 Preliminary Problem Statement 3
2.2 Product Position Statement 3
2.3 Stakeholders and Users 4
2.4 Scope and Key Features 4
3 Proposed Solution 6
3.1 Preliminary Architecture 6
3.2 Technology Stack 6
3.2.1 Core Technology Stack Summary 6
3.2.2 ORM Comparison Matrix 7
3.2.3 Hosting & Deployment Comparison Matrix 8
3.2.4 Testing Framework Comparison Matrix 9
3.3 Hardware and Operating Environment 10
4 Project Plan 12
4.1 Process and Phase Gates 12
4.2 Work Breakdown Structure (WBS) 14
4.3 Estimation 16
4.4 Schedule 22
4.5 Roles and Responsibilities 24
4.6 Human Resources and Cost Estimate 24
5 Risk Management 25
6 Tools & Collaboration Setup 27
7 AI Usage Log 29

Project Proposal & Plan

Objectives

This document focus on the following topics:

- ✔ Completing the Project Proposal & Plan document with the following sections:
  - Vision
  - Proposed Solution
  - Project Plan
  - Risk Management
  - Tools & Collaboration Setup
- ✔ Including an AI usage log for this phase
- ✔ Understanding the Project Proposal & Plan document.

1 Member Contribution Assessment

ID	Name	Role(s)	Contribution (%)	Evidence
24127462	Huỳnh Trung Ngạn	PM	25%	Build a work breakdown structure (WBS) Define project phases and exit criteria Set up communicationchannels Set up task board
24127175	Lê Đoàn Nhật Huy	BA	25%	Define the problem Identify requirements risks Identify target users and stakeholders Define the scope and must-have features
24127445	Trương Hiển Long	PM	25%	Set up collaboration tools Present project schedule Present work package effort estimation
24127053	Trần Hoàng Minh Khang	Designer	25%	Preliminary Architecture Core Technology Stack Summary Identify technology risks Propose the hosting / deployment environmen

2 Vision

2.1 Preliminary Problem Statement

Currently, instructors and students frequently manage classroom tasks using a fragmented combination of non-specialized communication channels (such as Zalo or Discord), shared spreadsheets, and emails. While traditional Learning Management Systems (LMS) like Moodle exist, they often suffer from obsolete user interfaces, bloated features, or critical functionalities locked behind paywalls.

Vible aims to solve this by providing a lightweight, modern web hub (built as a Single-Page Application and Progressive Web App). It is designed to run smoothly on any modern web browser across desktop and mobile devices.

2.2 Product Position Statement

Element	Content
For	Teachers and students
Who	People who need an intuitive online platform to host/manage classroom activities and distribute learning materials
Vible	Is an Online Teaching Platform (or Learning Management System)
That	Provides an all-in-one hub to create courses, distribute materials, submit assignments, and provide grading feedback.
Unlike	Fragmented / Unspecilised chat apps (Zalo/Discord) or outdated traditional LMS platforms (Moodle).
Our product	Delivers a frictionless, lightweight SPA/PWA experience with a modern user interface and no unnecessary overhead.

2.3 Stakeholders and Users

#	Stakeholder / user	Description	Main interests
1	Teacher / Instructor	End user	Wants an intuitive, easy-to-use online platform to set up classes, distribute materials, assignments, and easily provide feedback to students.
2	Student	End user	Wants a reliable portal to access course materials managed by their teachers.

2.4 Scope and Key Features

ID	Feature	Description	Priority (M/S/C/W)
F01	Authentication & authorization	Verify users (login/registration) and enforce role-based access for Teachers and Students.	Must
F02	Course management	Allow teachers to create, update, and delete courses.	Must
F03	Enrollment & roster management	Allow teachers to add or remove students from course rosters.	Must
F04	Course material management	Allow teachers to post/delete course materials (announcements, text, links, files) and enrolled students to view/download them.	Must
F05	Assignment & submission management	Allow teachers to create assignments with deadlines and enrolled students to submit their work.	Must
F06	Grading & feedback	Allow teachers to grade submitted assignments and provide written feedback to students.	Should
F07	Announcements & discussion channels	Provide course-level channels for class announcements and Q&A discussions between teachers and students.	Should
F08	Notifications & reminders	Send in-app or email alerts for new materials, upcoming assignment deadlines, and graded work.	Could
F09	Profile customization	Allow users to customize their profile details (display name, bio, avatar).	Could

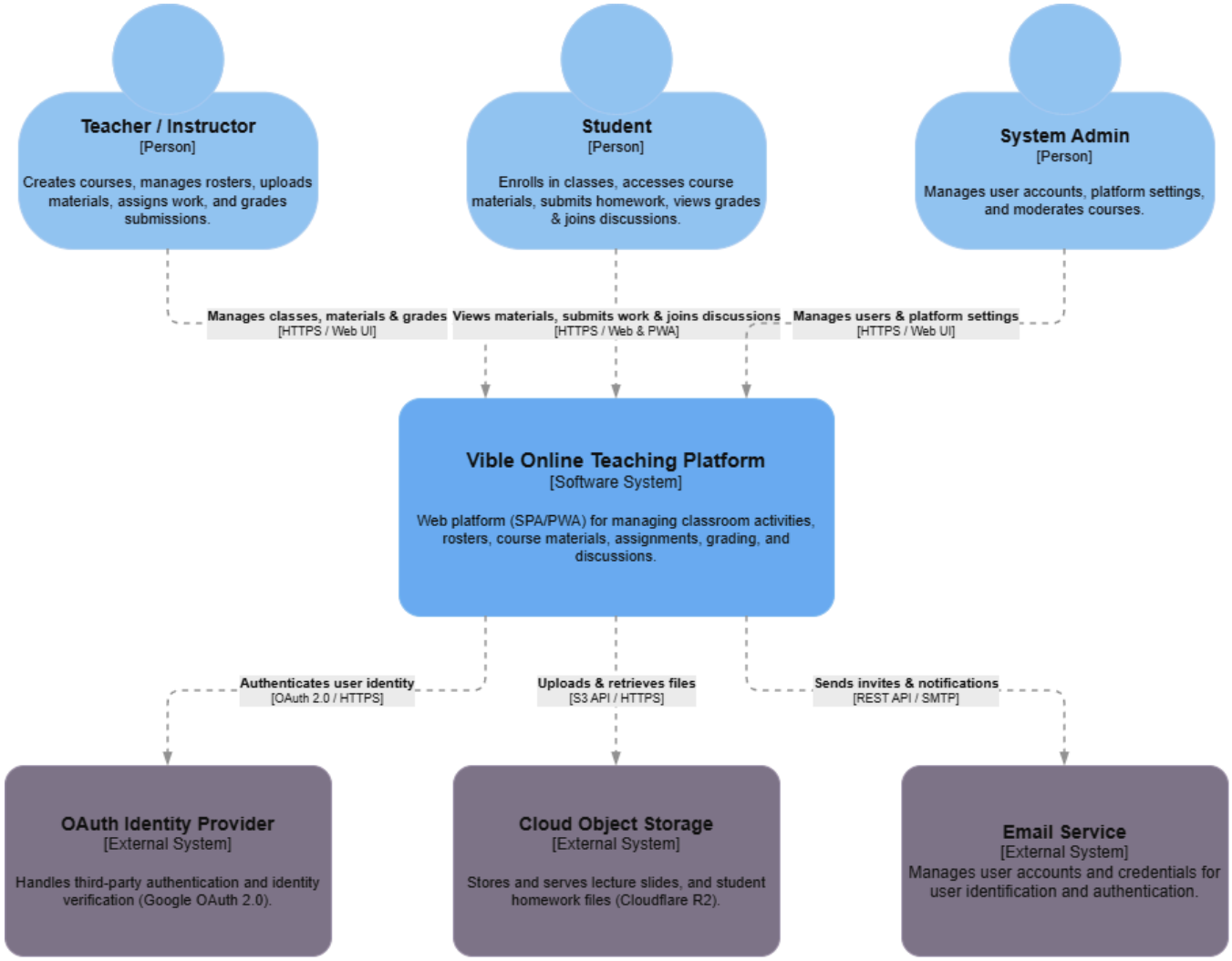
F10	Private in-app messaging	Real-time 1-on-1 private chat between students or teachers.	Won't
F11	Payment gateway integration	Real-money course checkout, billing, and subscription management.	Won't
F12	Live video conferencing	Built-in live video/audio streaming or virtual classroom calls.	Won't

Out of scope:

1. Real-time communication (F10, F12): Live video conferencing (e.g., Zoom/WebRTC integration) and real-time private 1-on-1 in-app chat.
2. Complex institutional administrative integrations: Institution-wide user directory syncing (LDAP/SAML) and bulk administrative roster imports.
3. Real-money payment gateway integration (F11): Commercial course checkout and payment processing.

3 Proposed Solution

3.1 Preliminary Architecture



3.2 Technology Stack

3.2.1 Core Technology Stack Summary

Layer	Technology	Reason for choice
Frontend	Vue, NuxtUI, TypeScript, Tailwindcss	**Vue**: Delivers a fast, responsive Single-Page Application (SPA) and Progressive Web App (PWA).

		Nuxt UI, Tailwindcss: provide highly customisable pre-built components, **TypeScript:** for type safety.
Backend	Nodejs, Nuxt (Fullstack)	Nuxt supports Typescript natively, handles things such as configuration, rendering, routing and many more. Nuxt offers powerful modules such as nuxt-ui, pinia and test-utils.
Database	PostgreSQL	Follows ANSI SQL standards, making standard queries transferable.
Hosting / deployment	Cloudflare	There are official presets for building Nuxt app on Cloudflare
Testing & CI	Nuxt test utils, ESlint, Prettier, GitHub Actions	**ESLint and Prettier**: enforces consistent code styling, formatting standards, and syntax validation across the codebase. **test-utils**: integrates the official Nuxt testing module to manage unit, component, and end-to-end (E2E) test suites, supporting test runners including Vitest, Jest, Playwright, and Cucumber. **GitHub Actions**: automates workflow pipelines to enforce quality gates—executing linting, formatting checks, and automated test suites.

3.2.2 ORM Comparison Matrix

Criteria	Drizzle ORM (Primary Choice)	Prisma ORM (Alternative Choice)
Bundle Size & Architecture	Ultra-lightweight (~7.4 KB, zero binary dependencies)	Relies on a heavy Rust Query Engine or WASM adapter
Cloudflare Edge Compatibility	100% Native on Cloudflare Workers/Pages (V8 Isolates) out of the box	Requires Serverless Driver Adapters or Prisma Accelerate to run on Cloudflare Edge
Cold Start Latency	Near-zero cold start	Slightly higher cold start due to engine initialization
Query Control & SQL Transparency	SQL-like TypeScript syntax; full control over JOINs and indexes (ideal for PA3)	High-level declarative API; abstracts SQL (risk of hidden N+1 queries)
Pros	· Seamless deployment on Cloudflare Pages/Workers · Zero runtime overhead & full TypeScript inference	· Beginner-friendly declarative schema.prisma syntax · Built-in Prisma Studio GUI for data inspection

	· Transparent SQL control for PA3 index optimization	· Mature migration CLI
Cons	· Smaller third-party plugin ecosystem than Prisma · Requires familiarity with SQL concepts	· Friction when bundling for Cloudflare V8 Isolates · Larger deployment bundle size
Verdict for Vible	Selected (Primary): Optimal compatibility with Cloudflare Edge and PostgreSQL.	Selected (Fallback/Local Dev): Supported via driver adapters if declarative schema is preferred.

3.2.3 Hosting & Deployment Comparison Matrix

Criteria	Cloudflare Pages / Workers (Selected)	Vercel (Hobby Tier)	Render (Free Tier)
Runtime Architecture	Global Edge Network (V8 Isolates)	Serverless Functions + Edge	Container / Native Node.js
Nuxt 4 / Nitro Support	First-class (cloudflare-pages preset & NuxtHub)	First-class (vercel preset)	Standard Node.js build
Free Tier Quotas	Unlimited static bandwidth; 100,000 Worker req/day	100 GB bandwidth/mo; strict serverless execution limits	100 GB bandwidth/mo; 750 instance hrs/mo
Cold Start / Spin-down	0 ms cold start (always warm at Edge)	~500 ms – 2 s cold start on inactive API routes	30 – 50 s spin-down after 15 mins of inactivity
Latency in Vietnam	Ultra-low (Local Vietnam Edge PoPs)	Low–Moderate (Singapore region)	Moderate (Singapore region)
Storage Synergy	Cloudflare R2 (10 GB free, $0 egress fee for course files)	Vercel Blob (very small free quota)	Paid persistent disks; free PostgreSQL expires in 30–90 days
Pros	· Zero spin-down ensures reliable TA grading & PA5 live demo · Unlimited static bandwidth & fast Vietnam latency · Free R2 object storage mitigates PA0 file-upload risk	· Effortless GitHub PR preview deployments · Native Node.js serverless compatibility	· Runs full Node.js runtime without V8 Isolate limits

Cons	· V8 Isolates restrict native C++ Node.js modules · 10 ms CPU time limit per free request	· Strict non-commercial & function execution caps	· 30–50 s cold spin-down severely degrades UX during evaluation
Verdict for Vible	Selected: Best free quotas, zero cold-start delay, and R2 storage integration.	Rejected: Tighter serverless quotas and higher latency than Cloudflare.	Rejected: 30–50 s spin-down delay risks failing NFR performance targets.

3.2.4 Testing Framework Comparison Matrix

Criteria	@nuxt/test-utils + Vitest (Selected - Unit/Int)	Jest (Unit/Int Alternative)	Playwright (Selected - System E2E)
Target Test Levels	Unit Testing & Component / API Integration Testing	Unit & Integration Testing	Automated System (E2E) & Acceptance Testing
Nuxt 4 & Vite Compatibility	Official Nuxt module; natively resolves auto-imports, Pinia stores, and Vue 3 SFCs	Poor; requires complex Babel/ESM transforms and manual Nuxt mocks	Integrated into @nuxt/test-utils; manages Nuxt server lifecycle automatically
Execution Speed	Ultra-fast (shares Vite's in-memory transformation pipeline)	Slow (heavy CommonJS resolution and full rebuilds)	Fast (parallel headless browser contexts with auto-wait)
Coverage & Reporting	Built-in V8/Istanbul coverage (@vitest/coverage-v8) for PA5 reports	Built-in Istanbul coverage	HTML reports, trace viewer, screenshots, and video recording
Pros	· Zero-config compatibility with Nuxt 4 & TypeScript · Fast feedback loop in local dev and GitHub Actions CI · Directly generates PA5 statement/branch coverage metrics	· Mature ecosystem with extensive legacy documentation	· Directly fulfills PA5 requirement (2 Use Cases × 2 E2E scenarios) · Multi-browser support (Chromium, Firefox, WebKit) & OAuth flows
Cons	· Full Nuxt runtime environment takes a few seconds to boot	· High maintenance overhead with modern ESM and Vite projects	· Requires downloading browser binaries (~200 MB) in CI
Verdict for Vible	Selected (Unit/Integration): Official,	Rejected: Incompatible with Vite/Nuxt 4 without brittle workarounds.	Selected (System E2E): Best automation tool for PA4/PA5 system testing.

	fastest, and most reliable runner for Nuxt 4.		

3.3 Hardware and Operating Environment

Hardware: Personal laptops/PCs equipped with at least a 4-core CPU (Intel Core i5 / AMD Ryzen 5 / Apple M-series or higher), 16 GB RAM (8 GB minimum), and 20 GB of free SSD storage.

Operating System: Windows 10/11 (with WSL2), macOS 13+, or Linux (Ubuntu 22.04 LTS+).

Software & Local Tooling: Node.js (v20 LTS / v22 LTS), npm/pnpm package manager, Git, Visual Studio Code (with Vue Official, ESLint, Prettier, and Tailwind CSS IntelliSense extensions), Wrangler CLI (for local Cloudflare Pages/Workers emulation), and Docker Desktop / local PostgreSQL 16 instance.

Application Hosting: Cloudflare Pages & Workers running the compiled Nuxt 4 Nitro server bundle across global Edge V8 Isolates over HTTPS (TLS 1.3) with automated CI/CD previews linked to GitHub.

Database & Object Storage: Managed serverless **PostgreSQL** instance (e.g., Neon / Supabase with connection pooling) for relational data and **Cloudflare R2** object storage bucket for course materials and student submission files.

Client / End-User Operating Environment (Teachers, Students & Admins):

Devices: Desktop PCs, laptops, tablets, and smartphones (supported via responsive Tailwind CSS/Nuxt UI layouts and Progressive Web App capabilities).

Web Browsers: Any modern Evergreen web browser supporting HTML5, CSS3, and ES6+, including Google Chrome (v115+), Mozilla Firefox (v115+), Microsoft Edge (v115+), and Apple Safari (v16+).

Network: Standard broadband or mobile internet connection (Wi-Fi / 4G / 5G) with HTTPS access (port 443).

4 Project Plan

4.1 Process and Phase Gates

The project follows the waterfall model. Each phase ends with a phase gate (a PA submission). After a PA is submitted it becomes a baseline; later changes must be updated in the document and listed in the next PA.

PA	Phase	Due	Exit criteria (fill in)
PA1	Initiation & planning	End of week 3	**Vision:** Outlined the problem, identified stakeholders, and determined key functionality scope. **Solution:** Finalized C4 System Context design, defined the core tech stack, and specified system operational environments. **Project Plan:** Constructed a WBS, calculated cost estimates and assigned member roles and responsibilities. **Risk Management:** Identify project risks. **Tools & Collaboration Setup:** github code repository, created the project taskboard, set up discord server as the primary discussion channel, and established status meeting schedules.
PA2	Requirements	End of week 5	**Requirements:** Defined functional, non-functional and domain requirments. **Use Case Model:** Finalized the complete use case diagram, defined actors and use case specifications. **Prototype / Mockups**: Present the screen list and wireframes/mockups for the main use cases **Traceability & Review:** Established the requirements matrix.

PA3	Design	End of week 7	**Architectural Design:** Constructed the architectural design with required diagrams and specifications. **Object-Oriented Design:** Constructed object-oriented design with required diagrams and specifications. **Data Design:** Constructed data design with required diagrams and specifications. **UI/UX Design:** Identifed how users will interact with the application and provided required diagrams and specifications.
PA4	Test planning	End of week 8	**Test Plan**: Defined testing scope, levels, environment, entry/exit criteria, and defect management. **Test Design**: Applied test design techniques (EP, BVA, and decision tables) to key features. **Test Cases**: Specified repeatable test cases for main workflows. **Traceability**: Mapped test cases to Must-have requirements in the traceability matrix.
PA5	Implementation, testing & deployment	End of week 10	**Implementation:** Implemented Must-have features with reviewed source code and logged design deviations. **Testing & CI:** Executed test cases, recorded results and defects, and configured automated tests with CI. **Deployment & Working Software:** Deployed accessible working software and recorded the demo video. **Project Retrospective:** Completed plan-vs-actual evaluation, process reflection, and AI usage review.
Final	Presentation & demo	Week 11	

4.2 Work Breakdown Structure (WBS)

Phase	Deliverable	Work package
1. Initiation & planning	1.1 Vision	1.1.1 Define the problem 1.1.2 Identify target users and stakeholders 1.1.3 Define the scope and must-have features
	1.2 Proposed Solution	1.2.1 Propose the preliminary application architecture 1.2.2 Propose the technology stack 1.2.3 Propose the hosting / deployment environment
	1.3 Project Plan	1.3.1 Define project phases and exit criteria 1.3.2 Build a work breakdown structure (WBS) 1.3.3 Assign roles and responsibilities 1.3.4 Present project schedule 1.3.5 Present work package effort estimation
	1.4 Risk Management	1.4.1 Identify technology risks 1.4.2 Identify requirements risks
	1.5 Tools & Collaboration Setup	1.5.1 Set up communication channels 1.5.2 Set up task board 1.5.3 Set up collaboration tools
2. Requirements	2.1 Functional, non-functional and domain requirements	2.1.1 Define the functional requirements 2.1.2 Define the non-functional requirements 2.1.3 Assign priorities to the requirements 2.1.4 Define business rules
	2.2 Use case model	2.2.1 List use cases 2.2.2 List actors
	2.3 Prototype / mockups	2.3.1 Deliver a prototype / mockups
	2.4 Traceability	2.4.1 Make an RTM (Requirements Traceability Matrix)
3. Design	3.1 Architectural design	3.1.1 Decide on the architectural style 3.1.2 Draw the system context diagram (update from PA1) 3.1.3 Draw the container diagram 3.1.4 Draw the component diagram 3.1.5 Draw the deployment Diagram
	3.2 Object-oriented design	3.2.1 Draw the conceptual class diagram 3.2.2 Draw the design-level class diagram 3.2.3 Draw the sequence diagrams for the main flow 3.2.4 Draw the State Machine Diagrams 3.2.5 Define Design Principles and Patterns
	3.3 Data design	3.3.1 Draw the conceptual ERD of the stored data 3.3.2 Schema Design & Modeling

		3.3.3 List Data specifications for DBMS 3.3.4 Identify normal form and the indexes created for frequent queries
	3.4 UI/UX design	3.4.1 Draw the screen flow diagram (navigation & routing) 3.4.2 Design page views and layout (wireframes) 3.4.3 List UX requirements (Accessibility)
4. Test planning	4.1 Test plan	4.1.1 List features to be tested or not tested 4.1.2 Map each feature with a test technique 4.1.3 Specify test environment and test data requirements
	4.2 Test cases	4.2.1 List test cases for each feature/function
	4.3 Traceability	4.3.1 Map every Must-Have feature/requirement to >= 1 test case
5. Implementation, testing & deployment	5.1 Pre-implementation	5.1.1 Set up and verify the coding environment and toolings 5.1.2 Set up the testing environment 5.1.3 Document coding standards and conventions 5.1.4 Configure continuous integration pipeline
	5.2 Frontend Implementation	5.2.1 Implement navigation and routing (according to screen flow diagram) 5.2.2 Implement page views and layout (according to wireframes) 5.2.3 Verify / Implement accessibility (according to UX requirements) 5.2.4 Wire CTAs to backend API endpoints
	5.3 Database Implementation	5.3.1 Install database engine 5.3.2 Configure access controls and credentials 5.3.3 Implement database schema and tables (according to schema design) 5.3.4 Seed test data
	5.4 Backend Implementation	5.4.1 Implement authentication and role-based access control (F01) 5.4.2 Implement course management (F02) 5.4.3 Implement enrollment and roster management (F03) 5.4.4 Implement course material upload and retrieval (F04) 5.4.5 Implement assignment creation and submission (F05) 5.4.6 [Optional] Implement secondary features (F06–F09)
	5.5 Testing	5.5.1 Verify the testing environment and test data 5.5.2 Execute Unit Testing 5.5.3 Document the test results and defects 5.5.4 Set up automated system testing (at least 2 use cases)

	5.6 Deployment	5.6.1 Deploy the application to the deployment environment
	5.7 Finishing up	5.7.1 Verify that the application is working as expected 5.7.2 Document the project retrospective 5.7.3 Record a demo video 5.7.4 Make a slide deck for the final presentation

4.3 Estimation

Colu mn 1	Work package	Owner	Effort (person-hours)	Method / assumptions
1.1.1	Define the problem	Lê Đoàn Nhật Huy	2	Expert judgement: 1-day duration (~2 person-hours/day); concise 1-2 page preliminary problem statement.
1.1.2	Identify target users and stakeholders	Lê Đoàn Nhật Huy	2	Expert judgement: 1-day duration; two primary end-user roles (Teacher, Student).
1.1.3	Define the scope and must-have features	Lê Đoàn Nhật Huy	4	Three-point estimate: O=2h, M=4h, P=6h -> E=4h (2-day duration); prioritizing 12 features (F01-F12) using MoSCoW.
1.2.1	Propose the preliminary application architecture	Trần Hoàng Minh Khang	2	Analogy: 1-day duration; similar to C4 System Context diagrams in previous coursework.
1.2.2	Propose the technology stack	Trần Hoàng Minh Khang	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); evaluating Nuxt 4, Drizzle vs Prisma, and Vitest/Playwright.
1.2.3	Propose the hosting / deployment environment	Trần Hoàng Minh Khang	2	Expert judgement: 1-day duration; comparing Cloudflare Pages/Workers, Vercel, and Render.
1.3.1	Define project phases and exit criteria	Huỳnh Trung Ngạn	4	Expert judgement: 2-day duration; defining exit criteria across 5 Waterfall phase gates (PA1-PA5).
1.3.2	Build a work breakdown structure (WBS)	Huỳnh Trung Ngạn	8	Three-point estimate: O=6h, M=8h, P=10h -> E=8h (4-day duration); decomposing 5 project phases into 3 levels.

1.3.3	Assign roles and responsibilities	Huỳnh Trung Ngạn	2	Expert judgement: 1-day duration; mapping 4 team members to primary and backup roles.
1.3.4	Present project schedule	Trương Hiển Long	4	Analogy: 2-day duration; building a 10-week Gantt chart and critical path similar to standard course templates.
1.3.5	Present work package effort estimation	Trương Hiển Long	4	Expert judgement: 2-day duration; calculating person-hours and documenting estimation methods for all WBS items.
1.4.1	Identify technology risks	Trần Hoàng Minh Khang	2	Expert judgement: 1-day duration; assessing Cloudflare V8 Isolate limits and PostgreSQL performance risks.
1.4.2	Identify requirements risks	Lê Đoàn Nhật Huy	2	Expert judgement: 1-day duration; assessing scope creep, estimation, and key member availability risks.
1.5.1	Set up communication channels	Huỳnh Trung Ngạn	2	Analogy: 1-day duration; standard Discord server setup and TA onboarding.
1.5.2	Set up task board	Huỳnh Trung Ngạn	4	Analogy: 2-day duration; configuring GitHub Projects board and creating issues for WBS work packages.
1.5.3	Set up collaboration tools	Trương Hiển Long	4	Analogy: 2-day duration; initializing GitHub repository, branch protection rules, and folder structure.
2.1.1	Define the functional requirements	Lê Đoàn Nhật Huy	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); specifying user requirements for F01-F09.
2.1.2	Define the non-functional requirements	Lê Đoàn Nhật Huy	4	Expert judgement: 2-day duration; specifying performance, security, and PWA/SPA compatibility targets.
2.1.3	Assign priorities to the requirements	Huỳnh Trung Ngạn	2	Expert judgement: 1-day duration; reviewing and refining MoSCoW priorities with stakeholders.
2.1.4	Define business rules	Lê Đoàn Nhật Huy	4	Expert judgement: 2-day duration; defining role permissions, enrollment constraints, and submission policies.
2.2.1	List use cases	Lê Đoàn Nhật Huy	8	Three-point estimate: O=6h, M=8h, P=10h -> E=8h (4-day duration); writing detailed use case specifications and flows.

2.2.2	List actors	Trần Hoàng Minh Khang	4	Analogy: 2-day duration; identifying actors and drawing the complete UML Use Case diagram.
2.3.1	Deliver a prototype / mockups	Trần Hoàng Minh Khang	8	Three-point estimate: O=6h, M=8h, P=10h -> E=8h (4-day duration); designing low/mid-fidelity wireframes for main workflows.
2.4.1	Make an RTM (Requirements Traceability Matrix)	Huỳnh Trung Ngạn	8	Analogy: 4-day duration; cross-referencing functional requirements, use cases, and design/test artifacts.
3.1.1	Decide on the architectural style	Trần Hoàng Minh Khang	2	Expert judgement: 1-day duration; documenting ADR for Nuxt 4 full-stack modular architecture on Edge.
3.1.2	Draw the system context diagram (update from PA1)	Trần Hoàng Minh Khang	4	Analogy: 2-day duration; refining PA1 C4 Level 1 diagram based on finalized PA2 SRS.
3.1.3	Draw the container diagram	Trần Hoàng Minh Khang	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); modeling C4 Level 2 (SPA, Nitro API, PostgreSQL, R2).
3.1.4	Draw the component diagram	Trần Hoàng Minh Khang	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); modeling C4 Level 3 backend and frontend modules.
3.1.5	Draw the deployment Diagram	Trương Hiển Long	4	Expert judgement: 2-day duration; mapping Cloudflare Pages/Workers, R2, and managed PostgreSQL nodes.
3.2.1	Draw the conceptual class diagram	Lê Đoàn Nhật Huy	4	Analogy: 2-day duration; domain modeling of core entities (User, Course, Enrollment, Material, Assignment).
3.2.2	Draw the design-level class diagram	Trần Hoàng Minh Khang	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); specifying methods, types, and relationships.
3.2.3	Draw the sequence diagrams for the main flow	Huỳnh Trung Ngạn	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); modeling interactions for key use cases.
3.2.4	Draw the State Machine Diagrams	Huỳnh Trung Ngạn	4	Analogy: 2-day duration; modeling lifecycle states for Courses and Assignment Submissions.

3.2.5	Define Design Principles and Patterns	Trương Hiển Long	4	Expert judgement: 2-day duration; specifying SOLID principles, repository/composable patterns, and middleware.
3.3.1	Draw the conceptual ERD of the stored data	Trương Hiển Long	4	Analogy: 2-day duration; entity-relationship modeling aligned with conceptual class diagram.
3.3.2	Schema Design & Modeling	Trương Hiển Long	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); defining Drizzle ORM PostgreSQL physical schema.
3.3.3	List Data specifications for DBMS	Trương Hiển Long	4	Expert judgement: 2-day duration; documenting column types, constraints, and foreign key policies.
3.3.4	Identify normal form and the indexes created for frequent queries	Trương Hiển Long	2	Expert judgement: 1-day duration; verifying 3NF/BCNF compliance and indexing foreign keys (mitigating Risk R5).
3.4.1	Draw the screen flow diagram (navigation & routing)	Trần Hoàng Minh Khang	4	Analogy: 2-day duration; mapping Nuxt page routing and role-based navigation guards.
3.4.2	Design page views and layout (wireframes)	Trần Hoàng Minh Khang	8	Three-point estimate: O=6h, M=8h, P=10h -> E=8h (4-day duration); high-fidelity UI layouts using Nuxt UI/Tailwind conventions.
3.4.3	List UX requirements (Accessibility)	Lê Đoàn Nhật Huy	2	Expert judgement: 1-day duration; defining WCAG contrast, keyboard navigation, and responsive design rules.
4.1.1	List features to be tested or not tested	Lê Đoàn Nhật Huy	2	Expert judgement: 1-day duration; scoping test coverage around Must-have features (F01-F05).
4.1.2	Map each feature with a test technique	Lê Đoàn Nhật Huy	2	Expert judgement: 1-day duration; assigning Equivalence Partitioning (EP), BVA, and Decision Tables.
4.1.3	Specify test environment and test data requirements	Trần Hoàng Minh Khang	6	Three-point estimate: O=4h, M=6h, P=8h -> E=6h (3-day duration); defining Vitest, Playwright, and local DB/R2 mock setups.

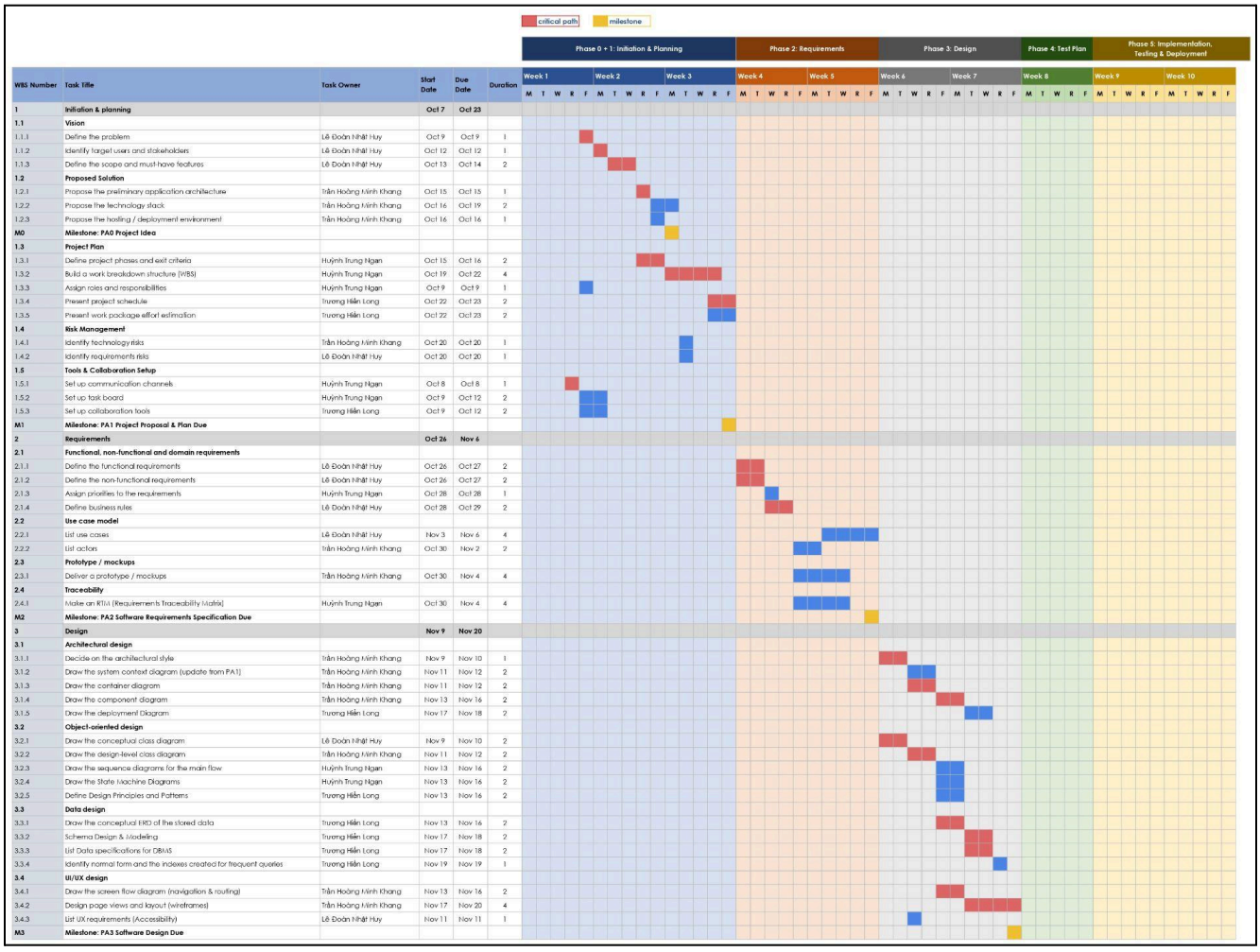
4.2.1	List test cases for each feature/function	Lê Đoàn Nhật Huy	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); writing repeatable test cases with preconditions and expected results.
4.3.1	Map every Must-Have feature/requirement to >= 1 test case	Huỳnh Trung Ngạn	2	Analogy: 1-day duration; updating RTM to link F01–F05 requirements to PA4 test cases.
5.1.1	Set up and verify the coding environment and toolings	Trương Hiển Long	2	Analogy: 1-day duration; initializing Nuxt 4, TypeScript, Tailwind CSS, and Wrangler CLI.
5.1.2	Set up the testing environment	Trương Hiển Long	2	Analogy: 1-day duration; configuring @nuxt/test-utils, Vitest, and Playwright runners.
5.1.3	Document coding standards and conventions	Huỳnh Trung Ngạn	2	Expert judgement: 1-day duration; configuring ESLint, Prettier, and Conventional Commits guidelines.
5.1.4	Configure continuous integration pipeline	Trương Hiển Long	2	Analogy: 1-day duration; setting up GitHub Actions workflow for automated linting and testing.
5.2.1	Implement navigation and routing (according to screen flow diagram)	Trần Hoàng Minh Khang	4	Analogy: 2-day duration; building Nuxt pages, layouts, and auth middleware.
5.2.2	Implement page views and layout (according to wireframes)	Trần Hoàng Minh Khang	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); building responsive components with Nuxt UI.
5.2.3	Verify / Implement accessibility (according to UX requirements)	Trần Hoàng Minh Khang	2	Expert judgement: 1-day duration; auditing ARIA attributes, semantic HTML, and keyboard focus.

5.2.4	Wire CTAs to backend API endpoints	Huỳnh Trung Ngạn	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); connecting frontend forms/views to Nitro API via useFetch/$fetch.
5.3.1	Install database engine	Trương Hiển Long	4	Analogy: 2-day duration; provisioning managed PostgreSQL (Neon/Supabase) and local Docker PostgreSQL 16.
5.3.2	Configure access controls and credentials	Trương Hiển Long	2	Expert judgement: 1-day duration; setting up environment variables, connection pooling, and role permissions.
5.3.3	Implement database schema and tables (according to schema design)	Trương Hiển Long	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); writing Drizzle ORM schema definitions and running migrations.
5.3.4	Seed test data	Lê Đoàn Nhật Huy	2	Analogy: 1-day duration; writing seed scripts for sample teachers, students, courses, and assignments.
5.4.1	Implement authentication and role-based access control (F01)	Trương Hiển Long	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); implementing session auth and Teacher/Student guards.
5.4.2	Implement course management (F02)	Huỳnh Trung Ngạn	4	Analogy: 2-day duration; standard CRUD API endpoints for courses.
5.4.3	Implement enrollment and roster management (F03)	Lê Đoàn Nhật Huy	4	Analogy: 2-day duration; API endpoints for adding/removing students in course rosters.
5.4.4	Implement course material upload and retrieval (F04)	Trương Hiển Long	8	Three-point estimate: O=6h, M=8h, P=10h -> E=8h (4-day duration); integrating Cloudflare R2 presigned URLs/buckets and metadata.
5.4.5	Implement assignment creation and submission (F05)	Huỳnh Trung Ngạn	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); handling deadline validation and student file/link submissions.

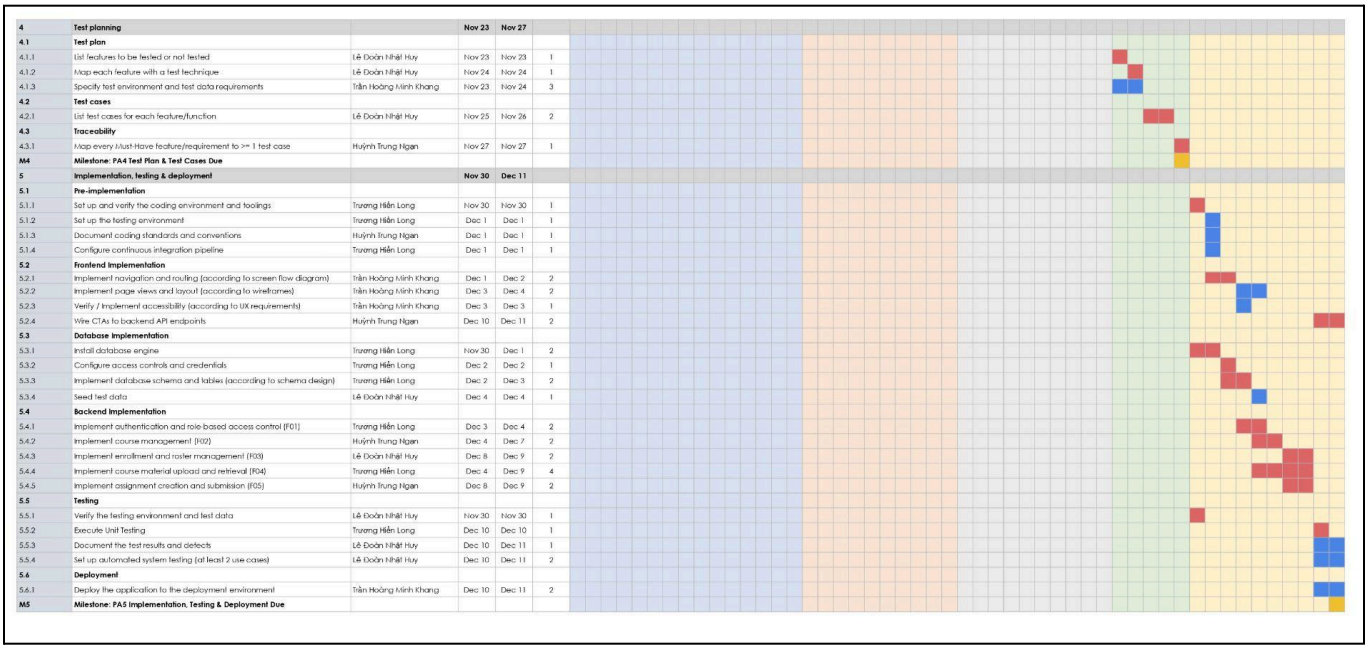
5.4.6	[Optional] Implement secondary features (F06-F09)	Unassigned (Optional)	0	Expert judgement: 0 baseline hours (Should/Could priority); implemented only if schedule buffer permits.
5.5.1	Verify the testing environment and test data	Lê Đoàn Nhật Huy	2	Expert judgement: 1-day duration; validating test database state before test execution.
5.5.2	Execute Unit Testing	Trương Hiển Long	2	Three-point estimate: O=1h, M=2h, P=3h -> E=2h (1-day duration); running Vitest suite and generating V8 coverage reports.
5.5.3	Document the test results and defects	Lê Đoàn Nhật Huy	2	Expert judgement: 1-day duration; logging pass/fail outcomes and tracking defects on GitHub Issues.
5.5.4	Set up automated system testing (at least 2 use cases)	Lê Đoàn Nhật Huy	4	Three-point estimate: O=3h, M=4h, P=5h -> E=4h (2-day duration); writing Playwright E2E tests for 2 main use cases.
5.6.1	Deploy the application to the deployment environment	Trần Hoàng Minh Khang	4	Three-point estimate: O=2h, M=4h, P=6h -> E=4h (2-day duration); deploying Nuxt 4 bundle to Cloudflare Pages/Workers and verifying R2/DB.
5.7.1	Verify that the application is working as expected	Trần Hoàng Minh Khang	2	Expert judgement: final production smoke testing on live Cloudflare URL.
5.7.2	Document the project retrospective	Huỳnh Trung Ngạn	2	Expert judgement: evaluating plan-vs-actual effort, schedule, and lessons learned.

4.4 Schedule

(*) find the spreedsheet @here



WBS Number	Task Title	Task Owner	Start Date	Due Date	Duration
1	**Initiation & planning**		Oct 7	Oct 23	
1.1	**Vision**				
1.1.1	Define the problem	Lê Đoàn Nhật Huy	Oct 9	Oct 9	1
1.1.2	Identify target users and stakeholders	Lê Đoàn Nhật Huy	Oct 12	Oct 12	1
1.1.3	Define the scope and must-have features	Lê Đoàn Nhật Huy	Oct 13	Oct 14	2
1.2	**Proposed Solution**				
1.2.1	Propose the preliminary application architecture	Trần Hoàng Minh Khang	Oct 15	Oct 15	1
1.2.2	Propose the technology stack	Trần Hoàng Minh Khang	Oct 16	Oct 19	2
1.2.3	Propose the hosting / deployment environment	Trần Hoàng Minh Khang	Oct 16	Oct 16	1
M0	**Milestone: PA0 Project Idea**				
1.3	**Project Plan**				
1.3.1	Define project phases and exit criteria	Huỳnh Trung Ngạn	Oct 15	Oct 16	2
1.3.2	Build a work breakdown structure (WBS)	Huỳnh Trung Ngạn	Oct 19	Oct 22	4
1.3.3	Assign roles and responsibilities	Huỳnh Trung Ngạn	Oct 9	Oct 9	1
1.3.4	Present project schedule	Trương Hiền Long	Oct 22	Oct 23	2
1.3.5	Present work package effort estimation	Trương Hiền Long	Oct 22	Oct 23	2
1.4	**Risk Management**				
1.4.1	Identify technology risks	Trần Hoàng Minh Khang	Oct 20	Oct 20	1
1.4.2	Identify requirements risks	Lê Đoàn Nhật Huy	Oct 20	Oct 20	1
1.5	**Tools & Collaboration Setup**				
1.5.1	Set up communication channels	Huỳnh Trung Ngạn	Oct 8	Oct 8	1
1.5.2	Set up task board	Huỳnh Trung Ngạn	Oct 9	Oct 12	2
1.5.3	Set up collaboration tools	Trương Hiền Long	Oct 9	Oct 12	2
M1	**Milestone: PA1 Project Proposal & Plan Due**				
2	**Requirements**		Oct 26	Nov 6	
2.1	**Functional, non-functional and domain requirements**				
2.1.1	Define the functional requirements	Lê Đoàn Nhật Huy	Oct 26	Oct 27	2
2.1.2	Define the non-functional requirements	Lê Đoàn Nhật Huy	Oct 26	Oct 27	2
2.1.3	Assign priorities to the requirements	Huỳnh Trung Ngạn	Oct 28	Oct 28	1
2.1.4	Define business rules	Lê Đoàn Nhật Huy	Oct 28	Oct 29	2
2.2	**Use case model**				
2.2.1	List use cases	Lê Đoàn Nhật Huy	Nov 3	Nov 6	4
2.2.2	List actors	Trần Hoàng Minh Khang	Oct 30	Nov 2	2
2.3	**Prototype / mockups**				
2.3.1	Deliver a prototype / mockups	Trần Hoàng Minh Khang	Oct 30	Nov 4	4
2.4	**Traceability**				
2.4.1	Make an RTM (Requirements Traceability Matrix)	Huỳnh Trung Ngạn	Oct 30	Nov 4	4
M2	**Milestone: PA2 Software Requirements Specification Due**				
3	**Design**		Nov 9	Nov 20	
3.1	**Architectural design**				
3.1.1	Decide on the architectural style	Trần Hoàng Minh Khang	Nov 9	Nov 10	1
3.1.2	Draw the system context diagram (update from PA1)	Trần Hoàng Minh Khang	Nov 11	Nov 12	2
3.1.3	Draw the container diagram	Trần Hoàng Minh Khang	Nov 11	Nov 12	2
3.1.4	Draw the component diagram	Trần Hoàng Minh Khang	Nov 13	Nov 16	2
3.1.5	Draw the deployment Diagram	Trương Hiền Long	Nov 17	Nov 18	2
3.2	**Object-oriented design**				
3.2.1	Draw the conceptual class diagram	Lê Đoàn Nhật Huy	Nov 9	Nov 10	2
3.2.2	Draw the design-level class diagram	Trần Hoàng Minh Khang	Nov 11	Nov 12	2
3.2.3	Draw the sequence diagrams for the main flow	Huỳnh Trung Ngạn	Nov 13	Nov 16	2
3.2.4	Draw the State Machine Diagrams	Huỳnh Trung Ngạn	Nov 13	Nov 16	2
3.2.5	Define Design Principles and Patterns	Trương Hiền Long	Nov 13	Nov 16	2
3.3	**Data design**				
3.3.1	Draw the conceptual ERD of the stored data	Trương Hiền Long	Nov 13	Nov 16	2
3.3.2	Schema Design & Modeling	Trương Hiền Long	Nov 17	Nov 18	2
3.3.3	List Data specifications for DBMS	Trương Hiền Long	Nov 17	Nov 18	2
3.3.4	Identify normal form and the indexes created for frequent queries	Trương Hiền Long	Nov 19	Nov 19	1
3.4	**UI/UX design**				
3.4.1	Draw the screen flow diagram (navigation & routing)	Trần Hoàng Minh Khang	Nov 13	Nov 16	2
3.4.2	Design page views and layout (wireframes)	Trần Hoàng Minh Khang	Nov 17	Nov 20	4
3.4.3	List UX requirements (Accessibility)	Lê Đoàn Nhật Huy	Nov 11	Nov 11	1
M3	**Milestone: PA3 Software Design Due**				



4	**Test planning**		Nov 23	Nov 27	
4.1	**Test plan**				
4.1.1	List features to be tested or not tested	Lê Đoàn Nhật Huy	Nov 23	Nov 23	1
4.1.2	Map each feature with a test technique	Lê Đoàn Nhật Huy	Nov 24	Nov 24	1
4.1.3	Specify test environment and test data requirements	Trần Hoàng Minh Khang	Nov 23	Nov 24	3
4.2	**Test cases**				
4.2.1	List test cases for each feature/function	Lê Đoàn Nhật Huy	Nov 25	Nov 26	2
4.3	**Traceability**				
4.3.1	Map every Must-Have feature/requirement to >= 1 test case	Huỳnh Trung Ngạn	Nov 27	Nov 27	1
M4	**Milestone: PA4 Test Plan & Test Cases Due**				
5	**Implementation, testing & deployment**		Nov 30	Dec 11	
5.1	**Pre-implementation**				
5.1.1	Set up and verify the coding environment and toolings	Trương Hiển Long	Nov 30	Nov 30	1
5.1.2	Set up the testing environment	Trương Hiển Long	Dec 1	Dec 1	1
5.1.3	Document coding standards and conventions	Huỳnh Trung Ngạn	Dec 1	Dec 1	1
5.1.4	Configure continuous integration pipeline	Trương Hiển Long	Dec 1	Dec 1	1
5.2	**Frontend Implementation**				
5.2.1	Implement navigation and routing (according to screen flow diagram)	Trần Hoàng Minh Khang	Dec 1	Dec 2	2
5.2.2	Implement page views and layout (according to wireframes)	Trần Hoàng Minh Khang	Dec 3	Dec 4	2
5.2.3	Verify / Implement accessibility (according to UX requirements)	Trần Hoàng Minh Khang	Dec 3	Dec 3	1
5.2.4	Wire CTAs to backend API endpoints	Huỳnh Trung Ngạn	Dec 10	Dec 11	2
5.3	**Database Implementation**				
5.3.1	Install database engine	Trương Hiển Long	Nov 30	Dec 1	2
5.3.2	Configure access controls and credentials	Trương Hiển Long	Dec 2	Dec 2	1
5.3.3	Implement database schema and tables (according to schema design)	Trương Hiển Long	Dec 2	Dec 3	2
5.3.4	Seed test data	Lê Đoàn Nhật Huy	Dec 4	Dec 4	1
5.4	**Backend Implementation**				
5.4.1	Implement authentication and role-based access control (F01)	Trương Hiển Long	Dec 3	Dec 4	2
5.4.2	Implement course management (F02)	Huỳnh Trung Ngạn	Dec 4	Dec 7	2
5.4.3	Implement enrollment and roster management (F03)	Lê Đoàn Nhật Huy	Dec 8	Dec 9	2
5.4.4	Implement course material upload and retrieval (F04)	Trương Hiển Long	Dec 4	Dec 9	4
5.4.5	Implement assignment creation and submission (F05)	Huỳnh Trung Ngạn	Dec 8	Dec 9	2
5.5	**Testing**				
5.5.1	Verify the testing environment and test data	Lê Đoàn Nhật Huy	Nov 30	Nov 30	1
5.5.2	Execute Unit Testing	Trương Hiển Long	Dec 10	Dec 10	1
5.5.3	Document the test results and defects	Lê Đoàn Nhật Huy	Dec 10	Dec 11	1
5.5.4	Set up automated system testing (at least 2 use cases)	Lê Đoàn Nhật Huy	Dec 10	Dec 11	2
5.6	**Deployment**				
5.6.1	Deploy the application to the deployment environment	Trần Hoàng Minh Khang	Dec 10	Dec 11	2
M5	**Milestone: PA5 Implementation, Testing & Deployment Due**				

4.5 Roles and Responsibilities

Role	Main	Backup	Responsibilities
Team lead / Project manager	Huỳnh Trung Ngạn	Trương Hiển Long	Plan, assign tasks, weekly status report, contact point
Business analyst	Lê Đoàn Nhật Huy	Trần Hoàng Minh Khang	Elicit and document requirements, organise reviews
Designer / Architect	Trần Hoàng Minh Khang	Huỳnh Trung Ngạn	Architecture, class, data and UI design
Developers	Trương Hiển Long	Lê Đoàn Nhật Huy	Implement, unit test, review pull requests
Tester	Lê Đoàn Nhật Huy	Trần Hoàng Minh Khang	Test plan, test cases, system testing, test report

4.6 Human Resources and Cost Estimate

Human Resources (Personnel Expense): 1,548 USD (assuming $6/hour)

1. **Trần Hoàng Minh Khang:** 74 person-hours ($740)
2. **Trương Hiển Long:** 64 person-hours ($640)
3. **Huỳnh Trung Ngạn:** 62 person-hours ($620)
4. **Lê Đoàn Nhật Huy:** 58 person-hours ($580)

Tooling, Infrastructure & Deployment: $0 USD

5 Risk Management

ID	Risk	Type	Probability	Loss	RE	Mitigation / contingency	Owner
R1	Key member unavailable during PA5.	People	0.2	8	1.6	**Mitigation**: Maintain strict documentation, modular code, and PR reviews so knowledge is shared. **Contingency**: Reassign tasks to the designated backup role defined in Section 4.5	Huỳnh Trung Ngạn
R2	Cloudflare Workers V8 Isolate limitations (10ms CPU limit on free tier, no native C++ Node.js modules) break backend dependencies.	Technology	0.3	7	2.1	**Mitigation**: Use Edge-compatible libraries (e.g., Drizzle ORM, Web Crypto API) and test builds early via wrangler. **Contingency**: Migrate deployment to Vercel or Render using standard Node.js runtime presets.	Trần Hoàng Minh Khang
R3	Underestimation of effort required for PA3 design models (UML/C4 diagrams).	Estimation	0.4	5	2.0	**Mitigation**: Standardize modeling tools and review requirements early in PA2. **Contingency**: Reallocate development	Lê Đoàn Nhật Huy

						buffers to complete design tasks before coding begins.	
R4	Incomplete or changing requirements during PA2 affecting feature implementation in PA5.	Requirements	0.3	6	1.8	**Mitigation**: Conduct thorough BA reviews and prioritize features using MoSCoW methodology. **Contingency**: Descope Should/Could features to safeguard core Must-have deliverables.	Lê Đoàn Nhật Huy
R5	Database performance degradation or slow queries under concurrent load during PA5 demonstration.	Technology	0.2	6	1.2	**Mitigation**: Design normalized relational schema and index frequently queried foreign key fields. **Contingency**: Implement query caching and optimize Drizzle ORM query execution plans.	Trương Hiển Long

6 Tools & Collaboration Setup

One of the main goals of the project is to practise teamwork for professional software development. The use of tools is monitored and graded (teamwork component). The team must use:

- ✔ Moodle – announcements and PA submissions.
- ✔ Communication – Discord
- ✔ Task board – GitHub Projects
- ✔ Git repository – GitHub
  - main branch is protected; changes are merged through pull requests with at least one review
  - README (description, setup), .gitignore; never commit secrets (.env, API keys, passwords)
  - commit messages follow the Conventional Commits style (feat:, fix:, docs:, test:, ...)
- ✔ Weekly status meeting (15 minutes) and weekly status report (template provided), committed to /docs/management/weekly every week from week 2 to week 10.

Repository structure:

Folder	Content
/src	Source code
/docs/management	Project plan; weekly status reports in /docs/management/weekly
/docs/requirements	Vision, SRS, use cases, mockups
/docs/design	Architecture (C4), ADRs, UML models, UI design
/docs/test	Test plan, test cases, test reports
/docs/ai-usage	AI usage logs
/pa/PA1 … /pa/PA5	One sub-folder per PA submission (PDF + source files)

7 AI Usage Log

#	Member	Tool	Task / purpose	How the output was verified and changed
1	Trương Hiển Long	ChatGPT	Project Plan	Refined and strengthened ideas and report, clarified the proposed solution, and identified edge cases to challenge assumptions.
2	Huỳnh Trung Ngạn	Gemini	Help fill in 4.2 Work Breakdown Structure (WBS)	Content was written entirely by the author
3	Huỳnh Trung Ngạn	Gemini	Help fill in 4.1 Process and Phase Gates	Content was written entirely by the author
4	Trần Hoàng Minh Khang	Gemini	Draft comparison matrices (pros, cons, and technical criteria) for Database & ORM (PostgreSQL, MongoDB, Firebase, Drizzle, Prisma), Hosting (Cloudflare, Vercel, Render, Railway), and Testing (@nuxt/test-utils, Vitest, Jest, Playwright, Cypress) in Section 3.2.	Reviewed with Trương Hiển Long and Huỳnh Trung Ngạn; cross-checked technical constraints against our Nuxt 4 repository configuration (nuxt.config.ts, package.json) and finalized PostgreSQL + Drizzle/Prisma, Cloudflare, and Vitest.
5	Trần Hoàng Minh Khang	Gemini	Generate initial C4 System Context (Level 1) diagram structure (XML/Mermaid) and draft descriptions for Section 3.1 (Preliminary Architecture) and Section 3.3 (Hardware and Operating Environment).	Imported the diagram into draw.io (app.diagrams.net), manually refined the layout, colors, actors (Teacher, Student, System Admin), and external systems (Google OAuth, Cloudflare R2, Email Service) to match PA0 scope boundaries.
6	Trần Hoàng Minh Khang	Kimi	Brainstorm technology and organisational risks, probability/loss ratings, and mitigation strategies for Section 5 (Risk Management).	Verified and adjusted risk exposure scores and mitigation plans with the Project Manager (Huỳnh Trung Ngạn) to align with the 3-week implementation schedule and Cloudflare Edge runtime constraints.